Vélib’ project: raw data quality and cleaning

Group H. Kenza Gueddas, Adem Ben Njima, Lara Normand, Amine Ben Bachir, Adrien Labergère. 03.11.2026

Our question, made precise after peer review

How often does a Vélib’ station fail its users (no bike to rent, or no free dock to return) while no other station within 300 m of it can serve them instead, and does this differ between Paris and the surrounding communes?

Three points are now explicit, as Group I suggested. Distances are measured from station to station, not from the user, because user positions are not in the data and the user is already at a station when the problem happens. Renting and returning are studied separately: a station with no bikes can still take returns. The comparison unit is the commune, summarised as Paris versus the rest of the network. The 300 m is the spacing Vélib’ Métropole gives for Paris (velib-metropole.fr/communes/paris, accessed 11.10.2026); our sensitivity check at 200, 300 and 500 m is unchanged.

What we checked, and on what

Every figure below comes from 21.09, the complete day shipped in `raw_sample/`, so a reader can reproduce all of them with `python code/run_all.py`. Run on our full archive, the same scripts give the period-wide tables in `data/clean/`.

We assessed quality at five levels, from the collection down to the meaning of one field:

Level	What can go wrong	How we check	Script
Collection	missing captures, failed requests	HTTP log vs 288 expected per day	`coverage.py`
File content	a 200 response with incomplete content	every station, every field, every file	`validate.py`
Rows	duplicates, impossible values	counts vs capacity, exact duplicates	`clean.py`
Stations	stations that come and go, freeze or go silent	same-day station list, run lengths	`clean.py`
Meaning	a field that does not mean what we assumed	`last_reported` vs the counts	`sampling_test.py`

Findings

Problem	Evidence (21.09)	Effect on our question	Decision
Incomplete capture	22:50 UTC: 24 KB instead of 34 KB, `stationCode` and the mechanical / electric split missing for all 1,518 stations	none on availability: bike and dock counts are present	keep, flag `partial_capture`; `collect.sh` v2 checks content
Truncated or unreadable capture	none in 295 checked files	would distort a whole time step	drop any capture with fewer than 1,400 stations
Exact duplicates	none on one machine	will appear when the two VMs are merged	drop repeated `(station_id, fetched_at)`
Stations entering and leaving	one station leaves after 22.09 (Argenteuil, capacity 0); Concorde and Invalides (about 410 docks each) appear on 24.09 and rent on one day only (26.09)	a station that is not really part of the network would count as a failure or as an alternative	match each day with that day's station list; flag `temporary` and `event_station`; exclude both from the panel
Missing capacity	4 stations with capacity 0	`docks_unavailable` cannot be computed	capacity 0 set to unknown, station kept
Counts above capacity	0.85% of rows: bikes + docks exceed capacity	small; counts are what users see	flag `capacity_conflict`, keep the counts
Docks not usable	in 50.6% of rows, bikes + docks is below capacity (median gap: 1 dock)	broken docks; already reflected in free docks	computed as `docks_unavailable`, no change
Out of service	1.6% of rows; 26 stations out of service all day	a real failure for users	kept, counted on both sides
Silent stations	23 stations with `last_reported` older than 3 h at some point, mostly out of service	their counts may be stale	flag `silent`; decide per analysis
Frozen stations	3 stations in service with identical counts for 12 h or more (one full for 24 h)	a stuck sensor would read as a long failure	flag `frozen`, exclude those rows from the panel
`last_reported` misread	see next section	our planned deduplication would have removed most episodes	dropped the deduplication; keep every capture

The main correction: what `last_reported` is

On 29.09 we read `last_reported` as the moment the counts were measured. From that we concluded that stations report every half hour, that 91.6% of our rows repeated an earlier report, and that we should keep one row per distinct `last_reported`. Group I showed this reading was wrong, and our own check (`sampling_test.py`) confirms it:

Measure (21.09)	Value
Consecutive captures with unchanged `last_reported` where the counts changed	30.3%
Count changes that happen while `last_reported` is unchanged	90.5%
`last_reported` renewals per station per hour, night / day	1.05 / 1.12
Count changes per station per hour, night / day	0.60 / 5.05

`last_reported` renews about once an hour, day and night alike, while the counts follow the activity of the city. It behaves like an hourly signal from the station, not a measurement time. The 32 minutes we reported as a delay is simply the average age of an hourly timestamp, and the 91% of repeated values is 11 captures out of 12. We renamed the column `lr_age_min` (it was `lag_min`) so nobody reads it as a delay of the counts.

We then tested what our plan would have cost, by recounting episodes with the same rule (at least two consecutive kept captures) on coarser data:

Kept data	No-bike episodes	5-min episodes still visible	Share of time without a bike
Every capture (5 min)	2,630	100%	6.67%
One capture every 15 min	1,376	92.6%	6.65%
One capture every 60 min	416	48.7%	6.57%
One row per `last_reported` (our old plan)	436	52.4%	5.82%

Coarser sampling barely changes the share of time without a bike, but it erases episodes: at 60 minutes or with our

old deduplication, only about one episode in six is still counted and half are not seen at all. The deduplication also biases the share of time downwards (5.82% instead of 6.67%), because it keeps the first row of each hour. Since 56% of no-bike episodes last less than 30 minutes, this is exactly the part of the answer we would have lost. The returning side gives the same picture (`data/clean/sampling_sensitivity.csv`). This also answers Prof. Pascual's question on the 5-minute interval: it is the rate at which the counts actually move.

What we still cannot know: the counts are renewed at least every 5 minutes, but the feed does not tell us how long after an event they change, so a short delay may remain.

Cleaning procedure and why

Principle: we flag, we do not alter. The feed is our only source, so we have nothing to correct a count against. Rows are removed only when they carry no usable information (a truncated capture, an exact duplicate). Every other problem becomes a boolean column, and the analysis uses one column, `in_panel`, so each exclusion is visible and can be reversed.

1. **Captures.** Drop captures with fewer than 1,400 stations (the network has about 1,519). Keep partial captures: availability only needs the counts.
2. **Duplicates.** Drop repeated `(station_id, fetched_at)`. Needed once the two VMs are merged.
3. **Same-day station list.** Join each day's status with the station list of that same day (union of its 24 hourly lists for membership, the last one for coordinates and capacity), as Group I suggested. A station absent from that day's list is flagged.
4. **Temporary and event stations.** `temporary`: listed on fewer than half the days of the period. `event_station`: capacity of 100 docks or more. Regular stations stop at 78 docks; above that we only find Concorde, Invalides, "Station Tour de France" and the Vincennes racecourse. Presence alone would miss the first two, which stay listed for weeks while open one day. Both are excluded from the panel, as failing stations and as alternatives.
5. **Capacity.** Capacity 0 becomes unknown. `docks_unavailable = capacity - bikes - free docks`; negative values are flagged as conflicts, not corrected.
6. **Service and the two sides.** `can_rent`: installed, renting and at least one bike. `can_return`: installed, returning and at least one free dock. `state` keeps the descriptive categories (out of service, empty, full, ok). We still never write "broken": the feed cannot tell a broken station from one that stopped reporting.
7. **Silent.** `last_reported` older than 180 minutes, three missed hourly signals.
8. **Frozen.** Counts identical for 12 hours or more while in service. On 21.09 the longest unchanged run is under 6 h for 95% of stations, so 12 h leaves a wide margin before we call a quiet station stuck.
9. **Panel.** `in_panel` = in the same-day list, not temporary, not an event station, not frozen.

On 21.09 the panel keeps 435,910 of 437,184 rows (99.7%) and 1,515 stations. In it, a station cannot rent 6.7% of the time and cannot take a return 4.7% of the time.

Episodes (`episodes.py`) are built on every capture: a run of at least two consecutive captures where the station cannot rent (or cannot take a return). On 21.09: 2,602 no-bike and 1,639 no-dock episodes, plus 28 on each side that last the whole day, almost all at stations out of service. The median no-bike episode is observed for 25 minutes, the median no-dock one for 20. Durations are what we saw: the true episode can be up to 5 minutes longer at each end, and on a single day, episodes are cut at midnight.

Storage: first comparison

`storage_benchmark.py` loads the cleaned day into SQLite (primary key on station and time, index on time) and compares it with our date-partitioned Parquet read by DuckDB, on the three queries the analysis needs:

21.09, 437,184 rows	**SQLite**	**Parquet + DuckDB**
Size on disk	46.8 MB (8 columns)	1.8 MB (all 29 columns)
Q1: share of time each station cannot rent (full scan)	63 ms	10 ms
Q2: one station's timeline (one station)	0.4 ms	7 ms
Q3: the whole network at one capture (one moment)	4.6 ms	36 ms

Parquet is about 25 times smaller with more columns, and faster on scans, which is most of our analysis. SQLite wins the selective lookups thanks to its indexes. Q3 is what the "viable alternative" rule repeats for every capture, so we will run it once as a grouped query rather than as many lookups, and measure again on the whole period before choosing.

Still open

- The full-period run of the cleaning (on our archive, not shipped) and the merge of the two machines with `compare_vms.py`.

- The commune boundaries (IGN) and population (INSEE), joined in the analysis step. Population is used as context (stations per resident, density), never as a count of affected users.
- `collect.sh` version 2 checks the content of every file before archiving it; captures archived by version 1 are checked afterwards by `validate.py`.
- Demonstrations and riots in Paris after the 29.09 snapshot will appear in the data collected later (stations closed or damaged). The data analysed here is not affected. For the final project we will mark those days and report the results with and without them.